An Empirical Study of Housing Price Drivers and Model Selection for Price Prediction

DATA 2010 – Winter 2026

March 28, 2026

Baher Alabbar
Student ID: 8038376

1 Project Overview

Research question: This project aims to determine which housing characteristics most strongly influence residential sale prices and how effectively these factors can be used to predict prices. In particular, the goal is to identify the most important features, evaluate whether engineered features improve predictive performance, and compare different models based on their ability to generalize to unseen data.

Why it matters: Understanding housing price drivers is important for multiple stakeholders, including buyers, sellers, real estate professionals, and policymakers. Buyers can make more informed decisions, sellers can better price their homes, and analysts can gain insights into how different property characteristics contribute to value. Additionally, accurate prediction models are widely used in real estate platforms and financial decision-making systems.

Main results: The OLS model achieved strong and consistent performance across all datasets, indicating good generalization. Feature engineering provided only minor improvements, and more complex models did not outperform the linear approach. Overall, house quality, living area, and structural features were the most influential predictors of price.

2 Research Question

This project aims to answer the following questions:

- Which housing features have the strongest influence on residential sale prices?
- Do engineered features improve predictive performance compared to using only the original variables?
- Which predictive model achieves the lowest error on unseen data?

3 Data Source and Description

3.1 Where the data came from

- **Dataset name:** House Prices – Advanced Regression Techniques
- **Link:** https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques/overview
- **What each row represents:** One residential property sale in Ames, Iowa.
- **Key columns used:** `SalePrice`, `GrLivArea`, `OverallQual`, `GarageCars`, `Neighborhood`, `YearBuilt`, `KitchenQual`
- **Licensing/usage notes:** The dataset is publicly available on Kaggle and intended for educational and research use as part of a machine learning competition.

3.2 Basic dataset facts

Number of rows	1460
Number of columns	81
Time range (if any)	2006–2010
Missing values present?	Yes, in several columns

4 Data Cleaning and Feature Engineering

This section outlines the steps taken to transform the raw dataset into a clean, consistent, and model-ready format.

Before performing any cleaning or preprocessing, the dataset was split into training and test sets using a randomized 80/20 split. The training set, consisting of 1,168 observations and 81 features, was used for all data cleaning, feature engineering, and model development to avoid data leakage. The test set was held out for final evaluation. All preprocessing steps and feature selection methods were applied exclusively on the training data.

4.1 Data Cleaning

What was done:

- **Feature reduction:** Removed variables with high missingness, redundant variables, and identifier columns such as `Id`.
- **Low-variance filtering:** Dropped categorical variables where a single category dominated over 90% of observations.
- **Leakage prevention:** Removed variables not available at prediction time (e.g., `SaleType`, `SaleCondition`, `MoSold`, `YrSold`).
- **Model-based feature selection:** Applied Lasso regression followed by tree-based importance ranking to retain the most relevant predictors.
- **Handling missing values:** Replaced missing basement-related values with a new category (`NB`) and used median imputation for remaining missing values.
- **Data type correction:** Converted variables into appropriate numerical and categorical formats.
- **Duplicate check:** Verified that no duplicate rows were present.
- **Outlier treatment:** Removed extreme values using the IQR method.
- **Rare category removal:** Removed infrequent categorical levels to reduce sparsity.
- **Target transformation:** Applied a log transformation to `SalePrice` to reduce skewness.

Evidence of cleaning:

Stage	Shape
Original training data	(1168, 81)
After high missingness removal	(1168, 75)
After redundancy elimination	(1168, 66)
After identifier removal	(1168, 65)
After low-variance filtering	(1168, 52)
After leakage removal	(1168, 48)
After Lasso feature selection	(1168, 42)
After rare category removal	(1131, 17)
After numerical outlier removal	(1126, 17)
Final dataset	(1126, 17)

4.2 Feature Renaming

After the cleaning and feature selection steps, the remaining variables were renamed to improve clarity and readability in the subsequent analysis and modeling phases.

Original Name	New Name
`FullBath`	`Full_Bathrooms`
`MSZoning`	`Zoning_Class`
`ExterQual`	`Exterior_Quality`
`BsmtFullBath`	`Basement_Full_Bathrooms`
`BldgType`	`Building_Type`
`BsmtQual`	`Basement_Quality`
`OverallCond`	`Overall_Condition`
`TotRmsAbvGrd`	`Total_Rooms_Above_Ground`
`KitchenQual`	`Kitchen_Quality`
`HouseStyle`	`House_Style`
`YearBuilt`	`Year_Built`
`BsmtExposure`	`Basement_Exposure`
`GrLivArea`	`Above_Ground_Living_Area`
`OverallQual`	`Overall_Quality`
`GarageCars`	`Garage_Capacity`
`SalePrice (log)`	`Log_Sale_Price`

4.3 Feature Engineering

The goal of this step is to create new features that better capture how different aspects of a house affect its price. Instead of relying only on raw variables, these features combine or transform existing ones to reflect more meaningful relationships.

- **Quality_LivingArea:** Combines overall quality and living area to capture that larger high-quality homes tend to have higher prices.
- **LivingArea_per_Room:** Measures the average size of each room, capturing how efficiently space is distributed.
- **Total_Bathrooms:** Combines all bathrooms into a single feature to better represent overall functionality.
- **House_Age:** Converts year built into age, which is more meaningful for modeling.
- **Quality_per_Room:** Captures the average quality per room, distinguishing between large low-quality homes and smaller high-quality ones.
- **Garage_per_Area:** Adjusts garage capacity relative to house size to reflect its relative importance.

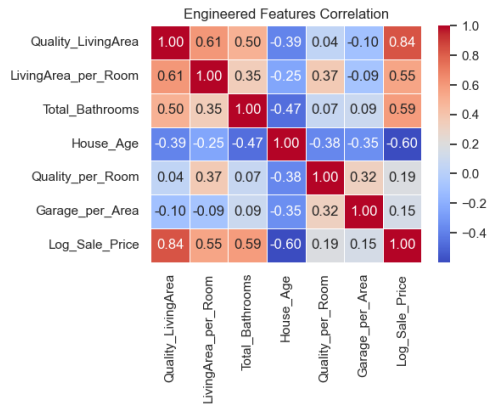

Figure 1: Correlation matrix of the engineered features and `Log_Sale_Price`. The plot highlights how the newly created features relate to the target variable and helps identify which ones capture stronger relationships.

5 Exploratory Data Analysis and Visualization

In this section, I focused on visualizing selected relationships, since examining all possible combinations of the 18 features would not be practical.

5.1 Plot 1: House Age vs Price

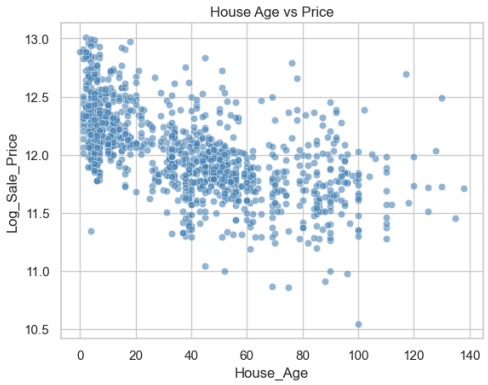

Figure 2: The relationship between house age and price shows a clear negative trend. As the age of a house increases, its price generally decreases. Older houses often require renovations and updates, which reduces their market value. In contrast, newer houses are typically designed to meet modern needs and standards, making them more desirable. This indicates that house age is an important predictor for price in the modeling stage.

5.2 Plot 2: Overall Quality vs Price

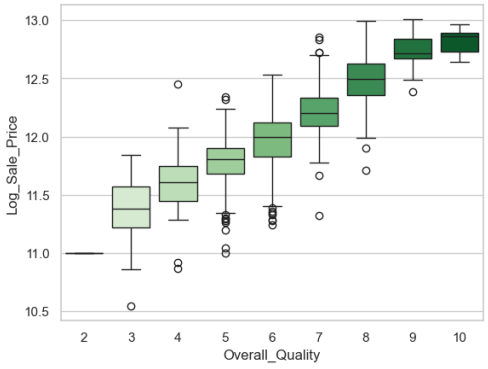

Figure 3: House quality has a strong and direct impact on price. As the overall quality increases, the price consistently increases as well. This relationship is very clear and expected, confirming that higher-quality houses are valued significantly more. This makes overall quality one of the most important features in predicting house prices.

5.3 Plot 3: Garage Capacity vs Price

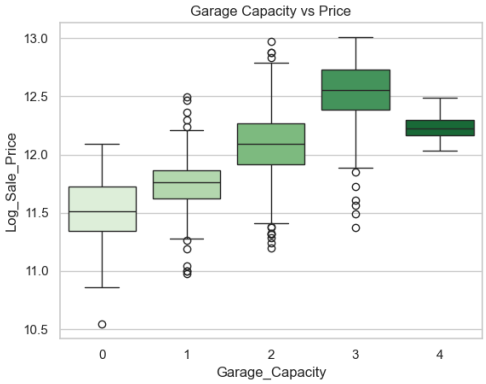

Figure 4: Garage capacity shows a generally increasing relationship with price at lower levels, where houses with more garage space tend to be more expensive. However, for very large garage capacities, this trend becomes less consistent and may even slightly decrease. This could be because extremely large garages are associated with specific use cases or property types that are not necessarily located in high-value residential areas.

5.4 Plot 4: Neighborhood vs Price

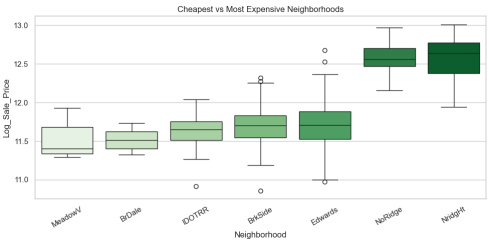

Figure 5: This plot compares house prices across selected neighborhoods representing different price ranges. Some neighborhoods share similar price distributions, while others are clearly higher or lower. This suggests that location plays a significant role in determining price, and that certain neighborhoods consistently contain more expensive properties.

5.5 Plot 5: Quality-Adjusted Living Area vs Price by Neighborhood

Figure 6: This plot shows that as the quality-adjusted living area increases, house prices increase as well. Additionally, the neighborhoods with higher prices also tend to have higher values of quality-adjusted living area. This indicates that expensive neighborhoods are not only higher in price due to location, but also due to having higher-quality houses. In contrast, lower-priced neighborhoods tend to have lower overall quality, reinforcing the combined effect of location and structural features on price.

6 Data Analysis

6.1 Analysis Plan

In this section, I began with an Ordinary Least Squares (OLS) regression model as a baseline to predict the target variable. This provided a simple and interpretable starting point for evaluating model performance. I then incorporated engineered features into the model to assess their impact and evaluate their contribution to improving predictions. Finally, I experimented with multiple models and different parameter settings to compare performance and determine the model that achieves the lowest prediction error.

6.2 Baseline

A linear regression (OLS) model was used as the baseline. The data was split into training (80%) and validation (20%) sets to evaluate performance on unseen data. The model showed similar results on both sets, indicating strong generalization and no evidence of overfitting.

6.3 Main Results

Training Performance

Model	RMSE	R^2
Baseline (OLS)	0.1209	0.8929
OLS + Engineered	0.1193	0.8957
Lasso	0.1194	0.8955
Ridge	0.1194	0.8956
XGBoost	0.0620	0.9718

Validation Performance

Model	RMSE	R^2
Baseline (OLS)	0.1198	0.8938
OLS + Engineered	0.1197	0.8939
Lasso	0.1202	0.8930
Ridge	0.1197	0.8938
XGBoost	0.1209	0.8917

All models were trained on the training set and evaluated on a held-out validation set to ensure a fair comparison and avoid overestimating performance.

6.4 Interpretation (So What?)

The results show that all linear-based models (OLS, Ridge, and Lasso) achieved nearly identical performance on both training and validation sets. The small gap between training and validation metrics indicates that these models generalize well and do not suffer from overfitting.

The inclusion of engineered features resulted in only a marginal improvement, suggesting that these features are largely redundant and do not add substantial new predictive information beyond the original variables.

In contrast, XGBoost achieved significantly better performance on the training set but worse performance on the validation set, indicating overfitting. This suggests that the dataset does not contain strong nonlinear patterns that more complex models can exploit.

Overall, the results indicate that the relationship between features and housing price is largely linear, and simpler models are sufficient for this task.

6.5 Final Model Selection

Based on the validation performance and model stability, the **OLS model without engineered features** is selected as the final model. It achieves the best validation performance while maintaining simplicity, interpretability, and strong generalization.

To further confirm this choice, the base OLS model was evaluated on the unseen test set, achieving an RMSE of 0.1540 and an R^2 of 0.8729. While slightly lower than the validation performance, these results remain consistent and indicate that the model generalizes well to new data. This reinforces the reliability of linear models for this dataset and supports the final selection.

7 Real-World Application

- **Stakeholder:** Home buyers, real estate agents, and property investors.
- **Decision:** Estimate a fair market price for a property based on its characteristics and compare properties across different neighborhoods and quality levels.
- **Recommendations:**
  - Prioritize features such as overall quality and living area, as they have the strongest impact on price.
  - Consider location (neighborhood) as a key factor when evaluating property value.
  - Use a simple linear regression model for price estimation, as it provides reliable predictions with high interpretability.
- **Confidence:** High. The consistency between training and validation performance across multiple models indicates that the results are stable and generalize well.

8 Report Quality and Reproducibility

8.1 How to run the code

- The full project is available on GitHub: https://github.com/b-4her/2010-house-price-prediction
- The main analysis is done in `EDA.ipynb`.
- To run the project:
  - Clone the repository or download it as a ZIP file.
  - Navigate to the project folder.
  - Install the required packages using `pip install -r requirements.txt`.
  - Open `EDA.ipynb` using Jupyter Notebook or VS Code.
  - Run all cells from top to bottom to reproduce the results.
- The project uses standard Python libraries including `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `seaborn`, and `xgboost`.

8.2 Project structure

- `README.md` provides a quick overview of the project and how to run it.
- `data/` contains the dataset files, including the train and test splits.
- `scripts/` includes the script used to split the dataset for easier analysis.
- `EDA.ipynb` contains all the code used for analysis and modeling.

9 Conclusion and Next Steps

Conclusion: Overall, features related to house quality, living area, and structural characteristics were the strongest drivers of sale price. Engineered features provided only marginal improvements, suggesting that the original variables already captured most of the predictive signal. Among the models tested, the OLS regression performed the best in terms of validation and test error, offering strong performance while maintaining simplicity and good generalization.

Next steps: If I had more time, I would experiment with a wider range of models, especially improving the neural network setup and tuning it more carefully. I would also restructure the project into modular components instead of keeping everything in a single notebook, making it easier to scale and maintain. Additionally, I would integrate the modeling process using an argument parser to efficiently test multiple configurations. Finally, I would deploy the trained models through an API, saving them using serialization so predictions can be made without retraining each time.

10 AI Disclosure

- **Tools used:** ChatGPT
- **What I used them for:** I mainly used ChatGPT to help me rephrase and organize parts of the report while writing. I also used it sometimes for brainstorming ideas, debugging parts of the code, and generating a few more complex code chunks that I later adjusted and integrated into the project.
- **Prompts I used:**
  - can you help me use lasso for feature selection on this dataset, like handle encoding + scaling properly and then return the most important original features not just the dummy columns, I want something I can explain in my report
  - I want to rank features using a model not just correlation, can you help me get feature importance using something like linear regression coefficients or tree based approach and then plot the top features clearly
  - can you help me write code to train a base OLS regression model on these selected house price features, split into train and validation, and print rmse and r2 for both so I can compare properly

– I added some engineered features like quality_livingarea and livingarea_per_room, can you help me update the linear regression code and compare it with the base model to see if they actually help or not
– can you give me clean code for lasso and ridge with cross validation, like use CV to pick alpha automatically and also show both train and validation rmse and r2 so I can analyze overfitting
– I trained xgboost and my train score is much higher than validation, does this make sense or am I doing something wrong, can you explain what is happening and maybe suggest better params
– can you help me preprocess the test set the same way as training, rename the columns, align the dummy columns with training columns, and then evaluate the final OLS model correctly
– my plots look a bit messy especially with many categories, can you help me clean them and maybe filter or reorder things so they are easier to read
– I have too many features to visualize all combinations, can you suggest a few meaningful plots that actually tell a story about price instead of random plots
– my validation results are very similar across models, is this normal or does it mean my features are already strong, just want to make sure im not missing something

References

- Dataset: House Prices - Advanced Regression Techniques, https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques/overview, accessed February 2026.
- Documentation: Scikit-learn, https://scikit-learn.org/stable
- Documentation: XGBoost, https://xgboost.readthedocs.io
- Documentation: Pandas, https://pandas.pydata.org/docs/
- Documentation: NumPy, https://numpy.org/doc/
- Documentation: Matplotlib, https://matplotlib.org/stable/contents.html
- Documentation: Seaborn, https://seaborn.pydata.org/
- Video: StatQuest – Decision Trees, https://www.youtube.com/watch?v=_L39rN6gz7Y
- Video: StatQuest – Regularization, https://www.youtube.com/watch?v=NGf0voTMlcs
- Video: StatQuest – XGBoost, https://www.youtube.com/watch?v=OtD8wVaFm6E